

Delegates in C#

Part-4

Lambda Expressions



Mr. Bangar Raju

 **NARESH**
technologies

Understanding Lambda Expressions in C#: A Simplified Approach to Anonymous Methods

By Albert Harmon

Published April 26, 2017

🕒 3 min read

Education



TLDR This blog post explains Lambda Expressions in C#, highlighting their role as a shorthand for anonymous methods. It covers the advantages of using Lambda Expressions, how they simplify code, and

provides examples to illustrate their implementation.

Lambda Expressions are a powerful feature in C# that simplify the use of anonymous methods. In this blog post, we will explore what Lambda Expressions are, how they relate to anonymous methods, and how to implement them effectively in your C# code.

What are Lambda Expressions?

Lambda Expressions were introduced in C# to provide a more concise way to write anonymous methods. An anonymous method is a method without a name that can be directly bound to a delegate. This allows for more flexible and readable code, especially when dealing with delegates.

The Concept of Anonymous Methods

In previous discussions about delegates, we learned about anonymous methods. An anonymous method allows you to define a method without specifying its name, return type, or access modifiers. This reduces the amount of code you need to write. For example, instead of defining a method like this:

```
public static string Greetings(string name) {  
    return "Hello " + name + ", a very good morning!";  
}
```

}

You can use an anonymous method directly with a delegate:

```
delegate string GreetingsDelegate(string name);

GreetingsDelegate greetings = delegate(string name) {
    return "Hello " + name + ", a very good morning!";
};
```

Advantages of Anonymous Methods

The primary advantage of using anonymous methods is the reduction in typing. You do not need to specify the access modifier, return type, or method name. You simply define the method body directly within the delegate.

Transition to Lambda Expressions

While anonymous methods were a significant improvement, they still required some boilerplate code, such as specifying the `delegate` keyword and the parameter types. This is where Lambda Expressions come into play.

What is a Lambda Expression?

A Lambda Expression is a shorthand way of writing anonymous methods. It uses the Lambda operator (``=>``) to separate the input parameters from the method body. This allows you to write cleaner and more concise code. For example, the previous anonymous method can be rewritten as a Lambda Expression:

```
GreetingsDelegate greetings = (name) => "Hello " + name + ", a
```



Key Features of Lambda Expressions

1. **No Need for Delegate Keyword:** You do not need to specify the ``delegate`` keyword when using Lambda Expressions.
2. **Parameter Types are Optional:** If the delegate type is known, you can omit the parameter types in the Lambda Expression.
3. **Simplified Syntax:** The syntax is more straightforward, making the code easier to read and maintain.

Implementing Lambda Expressions in C#

To illustrate how to implement Lambda Expressions, let's consider a simple example. Suppose we have a delegate defined as follows:

```
delegate string GreetingsDelegate(string name);
```

We can create a Lambda Expression that matches this delegate:

```
GreetingsDelegate greetings = (name) => "Hello " + name + ", a
```



Now, you can invoke this delegate just like before:

```
string result = greetings("Mr. Stick");  
Console.WriteLine(result);
```

This will output:

```
Hello Mr. Stick, a very good morning!
```

Conclusion

Lambda Expressions provide a simplified and more readable way to work with anonymous methods in C#. By reducing the amount of

boilerplate code and allowing for a more concise syntax, Lambda Expressions enhance the overall coding experience in C#. Understanding Lambda Expressions is essential for any C# developer looking to write cleaner and more efficient code.

For more insights and tutorials on C#, subscribe to our channel for the latest updates and videos.